

AhamX Bodhi

Where I Becomes Infinite

Language Model Head

Module 3.4: Language Model Head and Training Targets

Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- What the LM head is and where it sits in the architecture
- How hidden states become logits via a linear projection
- The role of softmax in producing a probability distribution
- Weight tying between the LM head and the embedding matrix
- How cross-entropy loss drives LM head training

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Domain:** Theory
- **Module:** 3.4 LM Head and Training Targets



Learning Objectives

By the End of This Concept You Will Be Able To

- **Explain** the purpose of the language model head in a decoder-only Transformer
- **Trace** the data flow from final hidden state to token probability distribution
- **Apply** the linear projection formula: $\text{logits} = \mathbf{h}\mathbf{W}_{LM}^T$
- **Describe** weight tying and justify why it reduces parameters without quality loss
- **Connect** cross-entropy loss to the training objective of the LM head



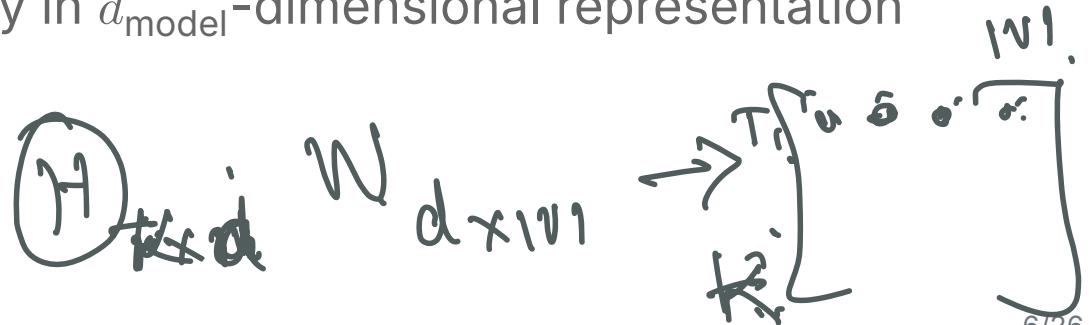


Recap: The Transformer Output Stack

The Final Three Layers of a Decoder-Only LLM

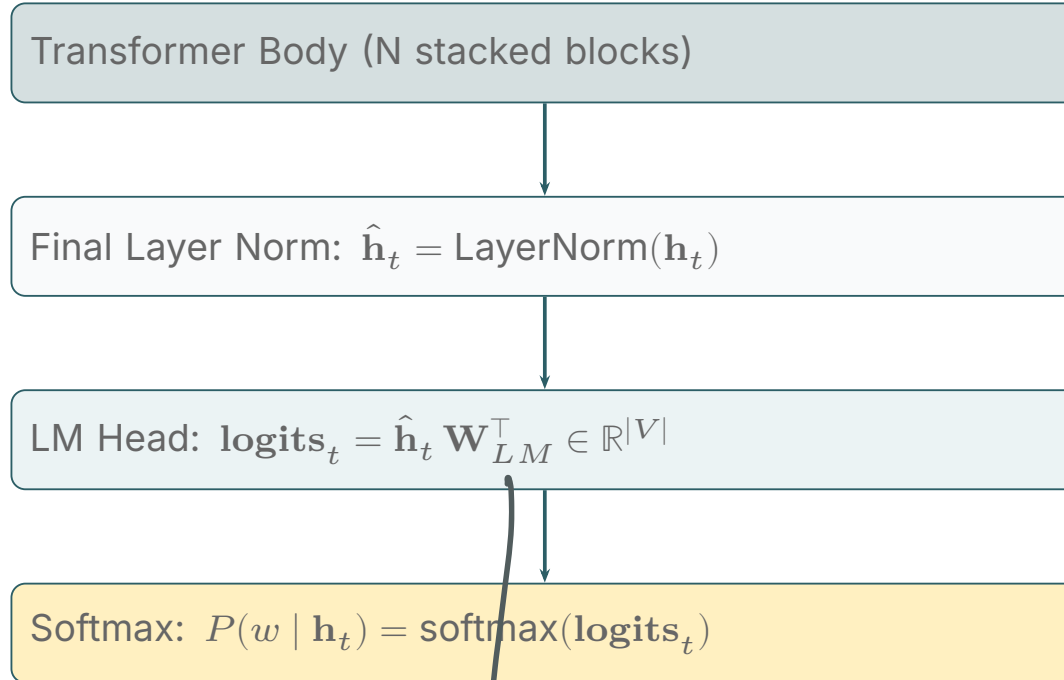
- The Transformer body produces a **final hidden state** $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ for each token position t
- After the last transformer block, a **final layer norm** stabilises the activations
- The **language model head** follows: a single linear layer that maps d_{model} dimensions to $|V|$ dimensions
- The result is a vector of **logits** — one real number per vocabulary token

The LM head is the only component that “knows” about the vocabulary. Everything below it operates purely in d_{model} -dimensional representation space.





LM Head: Data Flow Diagram



$$\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$$

$$\hat{\mathbf{h}}_t \in \mathbb{R}^{d_{\text{model}}}$$

$$\text{logits}_t \in \mathbb{R}^{|V|}$$

$$P \in \Delta^{|V|-1}$$

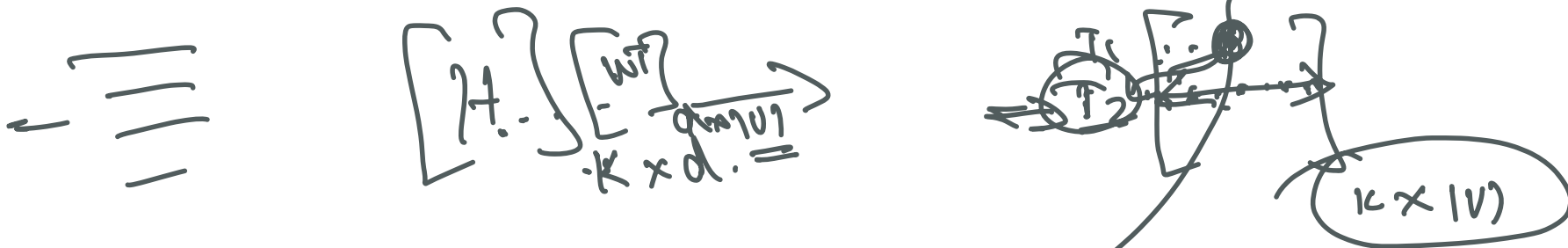
$L: |V|, m$



The Linear Projection: Hidden State to Logits

The LM Head Matrix $\mathbf{W}_{LM}$

- $\mathbf{W}_{LM} \in \mathbb{R}^{|V| \times d_{model}}$ — vocabulary size rows, model dimension columns
- For GPT-2 (small): $|V| = 50,257$, $d_{model} = 768 \Rightarrow \mathbf{W}_{LM}$ has $50,257 \times 768 \approx 38.6\text{M}$ parameters
- For Llama 3 8B: $|V| = 128,256$, $d_{model} = 4,096 \Rightarrow \mathbf{W}_{LM}$ has $\approx 525\text{M}$ parameters
- Each row of $\mathbf{W}_{LM}$ is a learned **output embedding** for one vocabulary token
- The dot product $\hat{\mathbf{h}}_t \cdot \mathbf{W}_{LM}^\top[v]$ measures how well the hidden state “matches” token v





From Logits to Probabilities: Softmax

The Softmax Transformation

- Logits are raw, unbounded real numbers — they are not probabilities
- Softmax converts them: $P(v | \mathbf{h}_t) = \frac{\exp(\text{logit}_v / T)}{\sum_{v'=1}^{|V|} \exp(\text{logit}_{v'} / T)}$
- Output is a proper probability distribution: all values in $(0, 1)$, sum to 1
- The highest logit becomes the highest probability token
- **Temperature scaling**: dividing logits by T before softmax controls sharpness

At inference time, only the **top-K** or **top-P** tokens are typically considered, so the full $|V|$ -dimensional softmax is often replaced with a truncated computation for efficiency.



Weight Tying: One Matrix, Two Roles



The Observation That Enables Weight Tying

- The **input embedding matrix** $\mathbf{E} \in \mathbb{R}^{|V| \times d_{\text{model}}}$ maps token IDs to vectors
- The **LM head matrix** $\mathbf{W}_{LM} \in \mathbb{R}^{|V| \times d_{\text{model}}}$ maps hidden states back to token scores
- Both have the same shape: both map between vocabulary space and model space
- **Weight tying** sets $\mathbf{W}_{LM} = \mathbf{E}$: the same matrix is used for both input and output
- Introduced in Press and Wolf (2017); now standard in GPT-2, GPT-J, Llama, and others

For Llama 3 8B this saves $\approx 525\text{M}$ parameters — nearly the size of a small standalone language model. Weight tying also provides a regularisation effect: tokens that are similar in input space must also be similar in output space.



Weight Tying: Properties and Trade-offs

Benefits

- Reduces model size by $|V| \times d_{\text{model}}$ parameters
- Improved gradient flow to the embedding layer
- Tokens frequent in input also improve output predictions
- Simpler to implement; single matrix to store

Limitations

- Input and output roles are not identical; some quality can be lost
- Not always optimal for very large vocabularies ($>100\text{K}$ tokens)
- Some large models (e.g., certain GPT-3 configurations) use separate matrices
- Untying is necessary when fine-tuning adds new tokens

M1: HB

2 for 10% for 1000

M2: SOB.

only a few
locations needed.
- 1. many

WT * F

2. 3

F

...

WT

2, 2, 4, 4 x 1

2 x 2 x 2 x 2 x 1

≈ 0.1% of total Params.



Training Objective: Causal Language Modelling

Next-Token Prediction as Self-Supervised Learning

- For a sequence $w_1, w_2, \dots, w_T$, the model must predict each w_t given $w_1, \dots, w_{t-1}$
- The training loss at position t is the **cross-entropy** between the predicted distribution and the true token:

$$\mathcal{L}_t = -\log P(w_t | w_1, \dots, w_{t-1}; \theta)$$

- The total loss is averaged over all positions: $\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{<t}; \theta)$
- No labels required — the next token in the training text **is** the label



Gradient Flow Through the LM Head

What the Loss Gradient Does

- The cross-entropy gradient with respect to logits is: $\nabla_{\text{logit}_v} \mathcal{L} = P(v) - \mathbf{1}[v = w_t]$
- This pushes the predicted probability of the *true* token up and ~~all other tokens~~ down
- With weight tying, this gradient also updates the input embedding for w_t
- Gradients flow back through the LM head into the Transformer body, training all layers jointly
- The LM head is the **sole source of supervision signal** for the entire model



Perplexity: Evaluating the LM Head

From Loss to Perplexity

- Perplexity (PPL) is the standard intrinsic metric for language model quality
- $\text{PPL} = \exp(\mathcal{L}) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{<t})\right)$
- Interpretation: average number of equally likely choices the model considers at each step
- Lower PPL = more confident and correct predictions = better model
- GPT-2 (117M) PPL on WikiText-103: ≈ 29.4 GPT-4: estimated < 5

Perplexity is computed *through* the LM head, making it a direct measure of how well the linear projection and weight tying together capture the statistical structure of language.





GenAI Application: Autoregressive Text Generation

The LM Head at Every Generation Step

- At each generation step, the LM head produces a $|V|$ -dimensional logit vector
- The sampling strategy (greedy, top-K, top-P, temperature) operates on these logits
- A single 128K-token generation request to Llama 3 70B invokes the LM head 128,000 times
- The LM head matrix multiplication dominates the output step: $O(d_{\text{model}} \times |V|)$ FLOPs per token
- Production serving systems (vLLM, TensorRT-LLM) fuse the LM head projection with the sampling kernel for speed

The LM head is on the critical path of every token generated. For Llama 3 8B, the LM head multiply alone is $4,096 \times 128,256 \approx 525\text{M}$ multiply-accumulate operations per token per request.



GenAI Application: Beyond Language --- Task-Specific Heads

Replacing the LM Head for Downstream Tasks

- **Classification head:** Replace $\mathbf{W}_{LM}$ with $\mathbf{W}_{cls} \in \mathbb{R}^{K \times d_{model}}$ for K classes
- **Token classification:** Per-token head for NER, POS tagging
- **Regression head:** Output a scalar for reward modelling (RLHF)

Production Examples

- BERT + classification head = sentiment analysis at Google Search
- GPT-2 + reward head = RLHF reward model in InstructGPT
- Llama + binary head = safety classifier in Llama Guard
- The Transformer body is reused; only the head changes



GenAI Application: Logit Manipulation for Control

Shaping Generation Through Logit Post-Processing

- **Logit bias (OpenAI API):** Add or subtract values from specific token logits to increase or ban certain words
- **Constrained decoding:** Libraries like Outlines and Guidance force outputs to match a JSON schema or regex by masking disallowed token logits to $-\infty$
- **Classifier-Free Guidance (CFG):** In diffusion-adapted LLMs, combine conditional and unconditional logits to amplify desired attributes
- **Speculative decoding:** A small draft model generates logits; the large model verifies them in parallel, reducing latency by 2–3×



GenAI Application: Fine-Tuning and the LM Head

LM Head Behaviour During Fine-Tuning

- Full fine-tuning updates all weights including the LM head and (if tied) the embeddings
- LoRA fine-tuning typically skips the LM head, only adapting attention and FFN layers
- Adding new domain tokens requires **untying weights** and initialising new rows in both embedding and LM head matrices
- RLHF training adds a **separate reward head** while keeping the LM head frozen during reward model training

When HuggingFace's `resize_token_embeddings()` is called to add new tokens, it extends both the embedding matrix and the LM head matrix simultaneously, preserving weight tying while accommodating the expanded vocabulary.



LM Head Efficiency in Production

The Memory Cost Problem

- The LM head matrix is the largest single weight matrix in many models
- Llama 3 8B LM head:
 $128,256 \times 4,096 \approx 525\text{M}$ parameters
- In FP16: ≈ 1 GB just for the LM head
- Weight tying avoids storing it twice

Optimisation Strategies

- **Flash-decode:** Fused LM head kernel reduces memory reads
- **INT8 / INT4 quantization** of LM head weights
- **Vocabulary pruning:** Remove rarely used tokens for domain models
- **Early exit:** Predict from intermediate layers at lower cost



Common Misconceptions

Myths and Corrections

- **Myth:** "The LM head is a complex neural network." It is a *single* linear layer (matrix multiply) with no activation function
- **Myth:** "Softmax is part of the LM head." Softmax is a separate post-processing step; the head itself outputs raw logits
- **Myth:** "Weight tying always hurts quality." Empirically it improves or matches quality in nearly all published ablations
- **Myth:** "The LM head is specific to generative models." Any model that produces a token distribution (masked LM, seq2seq) has an LM head

Key Takeaways



What You Have Learned

- The LM head is a single linear layer that maps the final hidden state to a $|V|$ -dimensional logit vector
- Softmax converts logits to a probability distribution; the next token is sampled from this distribution
- Weight tying sets the LM head matrix equal to the input embedding matrix, saving hundreds of millions of parameters
- Cross-entropy loss over next-token predictions is the sole training signal; gradients flow through the LM head to train the entire model
- Logit manipulation (bias, masking, constrained decoding) enables fine-grained control over generation
- Task-specific heads (classification, regression, reward) replace the LM head for downstream tasks

Thank You

Language Model Head — Module 3.4